Pneumonia Detection from Chest X-Rays

1. Imports and environment setup

In [1]:
```
from pathlib import Path
import os
import warnings
import random as rn
import glob

import cv2
import keras
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import plotly.graph_objects as go
import seaborn as sns
import tensorflow as tf
from keras.layers import BatchNormalization, Conv2D, Dense, Dropout, Flatten, Ma
from keras.models import Sequential
from sklearn.metrics import classification_report, confusion_matrix
from sklearn.model_selection import train_test_split
from tensorflow.keras import regularizers
from tensorflow.keras.applications.vgg16 import VGG16
from tensorflow.keras.callbacks import EarlyStopping, ModelCheckpoint, ReduceLRO
from tensorflow.keras.models import Model, load_model
from tensorflow.keras.optimizers import Adam, RMSprop, SGD
from tensorflow.keras.preprocessing import image
from tensorflow.keras.preprocessing.image import ImageDataGenerator
from tensorflow.keras.utils import to_categorical

warnings.filterwarnings("ignore")
plt.style.use("ggplot")
%matplotlib inline
```

2. Dataset loading and path configuration

In [ ]:
```
CURRENT_DIR = Path.cwd().resolve()
PROJECT_ROOT = CURRENT_DIR.parent if CURRENT_DIR.name == "notebook" else CURRENT

DATASET_ROOT = PROJECT_ROOT / "data" / "chest_xray" / "chest_xray"
TRAIN_DIR = DATASET_ROOT / "train"
VAL_DIR = DATASET_ROOT / "val"
TEST_DIR = DATASET_ROOT / "test"

MODELS_DIR = PROJECT_ROOT / "models"
RESULTS_DIR = PROJECT_ROOT / "results"



MODELS_DIR.mkdir(parents=True, exist_ok=True)
RESULTS_DIR.mkdir(parents=True, exist_ok=True)
```

```
print("Project root :", PROJECT_ROOT)
print("Dataset root :", DATASET_ROOT)
print("Train exists :", TRAIN_DIR.exists())
print("Validation exists :", VAL_DIR.exists())
print("Test exists :", TEST_DIR.exists())
```

```
Project root : C:\Users\user\Pneumonia_Detection
Dataset root : C:\Users\user\Pneumonia_Detection\data\chest_xray\chest_xray
Train exists : True
Validation exists : True
Test exists : True
```

In [7]:
```
for split_dir in [TRAIN_DIR, VAL_DIR, TEST_DIR]:
    print(f"\n{split_dir.name.upper()} directory:")
    if split_dir.exists():
        for class_dir in sorted(split_dir.iterdir()):
            if class_dir.is_dir():
                print(f"  - {class_dir.name}: {len(list(class_dir.glob('*')))} f
    else:
        print("  Directory not found.")
```

```
TRAIN directory:
  - NORMAL: 1092 files
  - PNEUMONIA: 3876 files

VAL directory:
  - NORMAL: 9 files
  - PNEUMONIA: 9 files

TEST directory:
  - NORMAL: 234 files
  - PNEUMONIA: 250 files
```

In [8]:
```
base_dir = DATASET_ROOT
```

In [9]:
```
print(base_dir.exists())  # Vérifie si le dossier principal du dataset existe
```

```
True
```

In [10]:
```
def get_data(data_dir):
    images = []
    labels = []
    img_size = 128
    valid_extensions = {".jpg", ".jpeg", ".png"}

    for label in ["NORMAL", "PNEUMONIA"]:
        path = Path(data_dir) / label
        if not path.exists():
            print(f"Le répertoire {path} n'existe pas.")
            continue

        class_num = 0 if label == "NORMAL" else 1
        for img_path in path.iterdir():
            if img_path.suffix.lower() not in valid_extensions:
                continue

            try:
                img_arr = cv2.imread(str(img_path), cv2.IMREAD_GRAYSCALE)
                if img_arr is None:
                    print(f"Impossible de charger l'image {img_path.name}.")
```

```
                    continue

                resized_arr = cv2.resize(img_arr, (img_size, img_size))
                images.append(resized_arr)
                labels.append(class_num)
            except Exception as e:
                print(f"Erreur lors du traitement de l'image {img_path.name}: {e

    return images, labels

train_images, train_labels = get_data(TRAIN_DIR)
test_images, test_labels = get_data(TEST_DIR)
val_images, val_labels = get_data(VAL_DIR)
```

In [11]:
```
print(f"Train set: {len(train_images)} images")
print(f"Validation set: {len(val_images)} images")
print(f"Test set: {len(test_images)} images")
```

```
Train set: 4966 images
Validation set: 16 images
Test set: 484 images
```

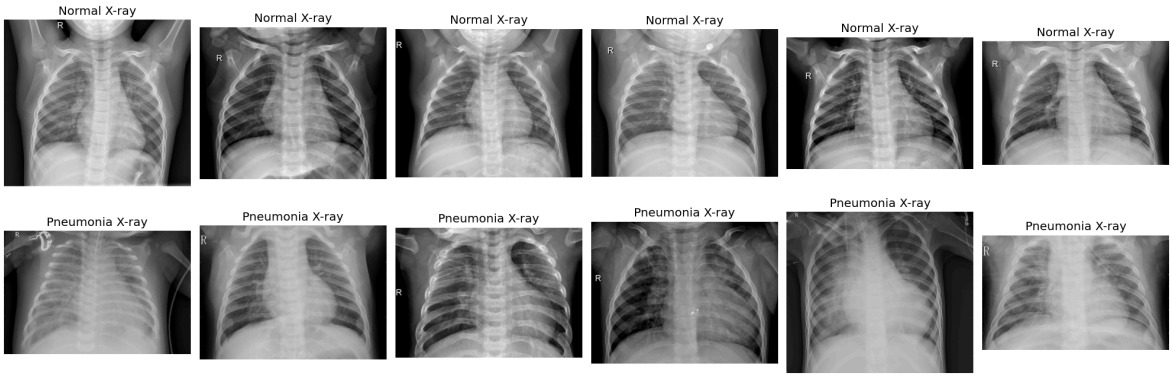
In [12]:
```
pneumonia_dir = TRAIN_DIR / "PNEUMONIA"
normal_dir = TRAIN_DIR / "NORMAL"

pneumonia = sorted([p.name for p in pneumonia_dir.iterdir() if p.is_file()])
normal = sorted([p.name for p in normal_dir.iterdir() if p.is_file()])

print(f"Nombre d'images de pneumonie : {len(pneumonia)}")
print(f"Nombre d'images normales : {len(normal)}")
```

```
Nombre d'images de pneumonie : 3876
Nombre d'images normales : 1092
```

3. Visualization and preprocessing

In [ ]:
```
VALID_EXTENSIONS = {".jpg", ".jpeg", ".png"}

normal_files = sorted([
    f for f in normal_dir.iterdir()
    if f.is_file() and f.suffix.lower() in VALID_EXTENSIONS
])

pneumonia_files = sorted([
    f for f in pneumonia_dir.iterdir()
    if f.is_file() and f.suffix.lower() in VALID_EXTENSIONS
])

plt.figure(figsize=(20, 10))

# Normal images
for i in range(6):
    plt.subplot(3, 6, i + 1)
    img = plt.imread(normal_files[i])
    plt.imshow(img, cmap="gray")
    plt.axis("off")
    plt.title("Normal X-ray")

# Pneumonia images
```

```
for i in range(6):
    plt.subplot(3, 6, i + 7)
    img = plt.imread(pneumonia_files[i])
    plt.imshow(img, cmap="gray")
    plt.axis("off")
    plt.title("Pneumonia X-ray")

plt.tight_layout()
plt.show()
```



In [16]:
```
import pandas as pd

# Fonction pour créer un DataFrame à partir des images et des étiquettes
def create_dataframe(images, labels, name):
    # Convertir les images en une liste de tableaux NumPy
    image_list = [img for img in images]

    # Créer un DataFrame
    df = pd.DataFrame({'image': image_list, 'label': labels})

    # Convertir les étiquettes en noms de classes
    df['label'] = df['label'].apply(lambda x: 'PNEUMONIA' if x == 1 else 'NORMAL

    print(f"{name} DataFrame créé avec {len(df)} lignes.")
    return df

# Créer les DataFrames
train_df = create_dataframe(train_images, train_labels, "Train")
test_df = create_dataframe(test_images, test_labels, "Test")
val_df = create_dataframe(val_images, val_labels, "Validation")

# Afficher les premières lignes des DataFrames
print("\nTrain DataFrame:")
print(train_df.head())

print("\nTest DataFrame:")
print(test_df.head())

print("\nValidation DataFrame:")
print(val_df.head())
```

```
Train DataFrame créé avec 4966 lignes.
Test DataFrame créé avec 484 lignes.
Validation DataFrame créé avec 16 lignes.

Train DataFrame:
                                                 image   label
0  [[22, 20, 16, 49, 60, 69, 74, 83, 90, 91, 96, ...  NORMAL
1  [[20, 26, 31, 35, 46, 45, 45, 53, 88, 90, 97, ...  NORMAL
2  [[0, 0, 17, 28, 37, 43, 54, 67, 71, 78, 82, 84...  NORMAL
3  [[42, 49, 55, 61, 71, 77, 78, 88, 90, 98, 101,...  NORMAL
4  [[82, 77, 54, 43, 28, 25, 22, 26, 26, 31, 33, ...  NORMAL

Test DataFrame:
                                                 image   label
0  [[36, 32, 30, 30, 28, 28, 29, 28, 28, 30, 27, ...  NORMAL
1  [[38, 33, 27, 64, 87, 102, 124, 138, 136, 147,...  NORMAL
2  [[39, 37, 34, 39, 40, 36, 33, 34, 34, 31, 32, ...  NORMAL
3  [[77, 74, 75, 80, 101, 94, 89, 83, 107, 122, 1...  NORMAL
4  [[39, 43, 39, 38, 34, 32, 33, 35, 36, 30, 27, ...  NORMAL

Validation DataFrame:
                                                 image   label
0  [[1, 11, 24, 34, 53, 66, 78, 99, 109, 111, 111...  NORMAL
1  [[2, 104, 91, 57, 4, 6, 6, 7, 2, 0, 0, 0, 1, 0...  NORMAL
2  [[1, 78, 96, 104, 103, 116, 124, 128, 128, 135...  NORMAL
3  [[1, 149, 62, 34, 33, 23, 27, 22, 17, 22, 15, ...  NORMAL
4  [[2, 13, 17, 14, 34, 40, 45, 50, 58, 62, 62, 6...  NORMAL
```

In [17]:
```
train_df
```

Out[17]:

	image	label
0	[[22, 20, 16, 49, 60, 69, 74, 83, 90, 91, 96, ...	NORMAL
1	[[20, 26, 31, 35, 46, 45, 45, 53, 88, 90, 97, ...	NORMAL
2	[[0, 0, 17, 28, 37, 43, 54, 67, 71, 78, 82, 84...	NORMAL
3	[[42, 49, 55, 61, 71, 77, 78, 88, 90, 98, 101,...	NORMAL
4	[[82, 77, 54, 43, 28, 25, 22, 26, 26, 31, 33, ...	NORMAL
...	...	...
4961	[[0, 0, 0, 0, 22, 34, 50, 56, 60, 69, 72, 80, ...	PNEUMONIA
4962	[[91, 92, 97, 91, 88, 111, 111, 97, 79, 82, 83...	PNEUMONIA
4963	[[27, 24, 11, 3, 0, 0, 0, 45, 46, 38, 48, 57, ...	PNEUMONIA
4964	[[29, 42, 35, 37, 37, 59, 53, 47, 47, 45, 46, ...	PNEUMONIA
4965	[[15, 12, 6, 7, 7, 6, 22, 30, 33, 36, 37, 37, ...	PNEUMONIA

4966 rows × 2 columns

In [18]:
```
test_df
```

Out[18]:

	image	label
0	[[36, 32, 30, 30, 28, 28, 29, 28, 28, 30, 27, ...	NORMAL
1	[[38, 33, 27, 64, 87, 102, 124, 138, 136, 147,...	NORMAL
2	[[39, 37, 34, 39, 40, 36, 33, 34, 34, 31, 32, ...	NORMAL
3	[[77, 74, 75, 80, 101, 94, 89, 83, 107, 122, 1...	NORMAL
4	[[39, 43, 39, 38, 34, 32, 33, 35, 36, 30, 27, ...	NORMAL
...	...	...
479	[[22, 47, 33, 46, 28, 207, 45, 200, 152, 30, 2...	PNEUMONIA
480	[[145, 85, 69, 28, 27, 11, 12, 16, 18, 17, 18,...	PNEUMONIA
481	[[22, 15, 10, 201, 198, 36, 6, 11, 10, 15, 30,...	PNEUMONIA
482	[[213, 41, 56, 53, 57, 58, 58, 60, 49, 158, 60...	PNEUMONIA
483	[[12, 16, 19, 27, 29, 37, 48, 54, 59, 59, 61, ...	PNEUMONIA

484 rows × 2 columns

In [19]:
```
val_df
```

Out[19]:

	image	label
0	[[1, 11, 24, 34, 53, 66, 78, 99, 109, 111, 111...	NORMAL
1	[[2, 104, 91, 57, 4, 6, 6, 7, 2, 0, 0, 0, 1, 0...	NORMAL
2	[[1, 78, 96, 104, 103, 116, 124, 128, 128, 135...	NORMAL
3	[[1, 149, 62, 34, 33, 23, 27, 22, 17, 22, 15, ...	NORMAL
4	[[2, 13, 17, 14, 34, 40, 45, 50, 58, 62, 62, 6...	NORMAL
5	[[4, 251, 60, 64, 68, 146, 117, 110, 111, 118,...	NORMAL
6	[[7, 30, 31, 37, 37, 43, 43, 43, 44, 39, 34, 2...	NORMAL
7	[[0, 0, 0, 6, 4, 8, 15, 16, 21, 22, 26, 23, 25...	NORMAL
8	[[17, 19, 20, 27, 33, 35, 47, 33, 55, 39, 42, ...	PNEUMONIA
9	[[2, 2, 1, 1, 1, 0, 243, 254, 2, 10, 11, 14, 1...	PNEUMONIA
10	[[11, 255, 33, 252, 61, 73, 88, 101, 105, 111,...	PNEUMONIA
11	[[0, 1, 13, 27, 28, 37, 53, 75, 87, 96, 103, 1...	PNEUMONIA
12	[[0, 0, 10, 19, 33, 56, 76, 85, 102, 105, 108,...	PNEUMONIA
13	[[0, 0, 0, 0, 0, 0, 1, 40, 1, 15, 17, 20, 25, ...	PNEUMONIA
14	[[63, 64, 63, 62, 64, 52, 253, 59, 64, 61, 62,...	PNEUMONIA
15	[[0, 0, 2, 3, 4, 9, 13, 18, 22, 28, 34, 42, 43...	PNEUMONIA

In [20]:
```
def plot_cases_count(df, title):
    cases_count = df["label"].value_counts().reindex(["PNEUMONIA", "NORMAL"])

    fig = go.Figure(
        data=[
            go.Bar(
                x=["PNEUMONIA", "NORMAL"],
                y=cases_count.values,
                marker_color=["lightblue", "orange"]
            )
        ]
    )

    fig.update_layout(
        title=title,
        xaxis_title="Classes",
        yaxis_title="Nombre d'images",
        font=dict(family="Century, sans-serif", size=16, color="grey"),
        template="plotly_white",
        hovermode="x unified",
        hoverlabel=dict(bgcolor="white", font_size=12),
    )
    return fig

fig_train = plot_cases_count(train_df, "Training Data")
fig_test = plot_cases_count(test_df, "Test Data")
fig_val = plot_cases_count(val_df, "Validation Data")

fig_train.show()
fig_test.show()
fig_val.show()
```

4. Data augmentation and resizing

In [21]:
```
from tensorflow.keras.applications.vgg16 import VGG16
from tensorflow.keras.callbacks import EarlyStopping, ModelCheckpoint, ReduceLRO
from tensorflow.keras.layers import Dense, Dropout, Flatten
from tensorflow.keras.models import Model
from tensorflow.keras.optimizers import Adam
from tensorflow.keras.preprocessing.image import ImageDataGenerator
```

In [22]:
```
train_datagen = ImageDataGenerator(
    rescale=1.0 / 255.0,  # Normalisation des pixels
    rotation_range=20,
    width_shift_range=0.2,
    height_shift_range=0.2,
    shear_range=0.2,
    zoom_range=0.2,
    horizontal_flip=True,
    fill_mode='nearest'
)

# Normalisation des données pour la validation et le test
valid_datagen = ImageDataGenerator(rescale=1.0 / 255.0)
test_datagen = ImageDataGenerator(rescale=1.0 / 255.0)
```

In [23]:
```
train_dir = TRAIN_DIR
val_dir = VAL_DIR
test_dir = TEST_DIR

train_generator = train_datagen.flow_from_directory(
    train_dir,
    target_size=(128, 128),
    batch_size=32,
    class_mode="categorical",
    shuffle=True,
    seed=42,
    color_mode="rgb"
)

valid_generator = valid_datagen.flow_from_directory(
    val_dir,
    target_size=(128, 128),
    batch_size=32,
    class_mode="categorical",
    shuffle=True,
    seed=42,
    color_mode="rgb"
)

test_generator = test_datagen.flow_from_directory(
    test_dir,
    target_size=(128, 128),
    batch_size=32,
    class_mode="categorical",
    shuffle=False,
    color_mode="rgb"
)
```

```
Found 4966 images belonging to 2 classes.
Found 16 images belonging to 2 classes.
Found 484 images belonging to 2 classes.
```

In [24]:
```
class_labels = train_generator.class_indices
class_labels
```

Out[24]:
```
{'NORMAL': 0, 'PNEUMONIA': 1}
```

In [25]:
```
class_name = {value:key for (key, value) in class_labels.items()}
class_name
```

Out[25]:
```
{0: 'NORMAL', 1: 'PNEUMONIA'}
```

5. VGG16 transfer learning architecture

In [26]:
```
from tensorflow.keras import regularizers
base_model = VGG16(
    input_shape=(128, 128, 3),  # Taille des images d'entrée
    include_top=False,          # Ne pas inclure les couches fully connected
    weights='imagenet'          # Utiliser les poids pré-entraînés sur ImageNet
)

# Geler les couches de base (ne pas les ré-entraîner initialement)
for layer in base_model.layers:
```

```
    layer.trainable = False

#Ajouter des couches personnalisées ( tête du réseau )
x = base_model.output
flat = Flatten()(x)
class_1 = Dense(1024, activation='relu', kernel_regularizer=regularizers.l2(0.01
dropout = Dropout(0.5)(class_1)
class_2 = Dense(512, activation='relu', kernel_regularizer=regularizers.l2(0.01)
output = Dense(2, activation='softmax')(class_2)

# Créer le modèle final
model_01 = Model(base_model.inputs, output)

# Compiler le modèle
model_01.compile(
    optimizer=Adam(learning_rate=1e-4),
    loss='categorical_crossentropy',
    metrics=['accuracy']
)

# Afficher un résumé du modèle
model_01.summary()
```

```
Downloading data from https://storage.googleapis.com/tensorflow/keras-application
s/vgg16/vgg16_weights_tf_dim_ordering_tf_kernels_notop.h5
58889256/58889256 ━━━━━━━━━━━━━━━━━━━━ 2s 0us/step
```

Model: "functional"

Layer (type)	Output Shape	Param #
input_layer (InputLayer)	(None, 128, 128, 3)	0
block1_conv1 (Conv2D)	(None, 128, 128, 64)	1,792
block1_conv2 (Conv2D)	(None, 128, 128, 64)	36,928
block1_pool (MaxPooling2D)	(None, 64, 64, 64)	0
block2_conv1 (Conv2D)	(None, 64, 64, 128)	73,856
block2_conv2 (Conv2D)	(None, 64, 64, 128)	147,584
block2_pool (MaxPooling2D)	(None, 32, 32, 128)	0
block3_conv1 (Conv2D)	(None, 32, 32, 256)	295,168
block3_conv2 (Conv2D)	(None, 32, 32, 256)	590,080
block3_conv3 (Conv2D)	(None, 32, 32, 256)	590,080
block3_pool (MaxPooling2D)	(None, 16, 16, 256)	0
block4_conv1 (Conv2D)	(None, 16, 16, 512)	1,180,160
block4_conv2 (Conv2D)	(None, 16, 16, 512)	2,359,808
block4_conv3 (Conv2D)	(None, 16, 16, 512)	2,359,808
block4_pool (MaxPooling2D)	(None, 8, 8, 512)	0
block5_conv1 (Conv2D)	(None, 8, 8, 512)	2,359,808
block5_conv2 (Conv2D)	(None, 8, 8, 512)	2,359,808
block5_conv3 (Conv2D)	(None, 8, 8, 512)	2,359,808
block5_pool (MaxPooling2D)	(None, 4, 4, 512)	0
flatten (Flatten)	(None, 8192)	0
dense (Dense)	(None, 1024)	8,389,632
dropout (Dropout)	(None, 1024)	0
dense_1 (Dense)	(None, 512)	524,800
dense_2 (Dense)	(None, 2)	1,026

Total params: 23,630,146 (90.14 MB)

Trainable params: 8,915,458 (34.01 MB)

Non-trainable params: 14,714,688 (56.13 MB)

In [27]:
```
checkpoint_path = MODELS_DIR / "best_model.keras"
```

```
es = EarlyStopping(monitor="val_loss", verbose=1, mode="min", patience=4)
cp = ModelCheckpoint(
    filepath=checkpoint_path,
    monitor="val_loss",
    save_best_only=True,
    save_weights_only=False,
    mode="auto",
    save_freq="epoch",
)
lrr = ReduceLROnPlateau(
    monitor="val_accuracy",
    patience=3,
    verbose=1,
    factor=0.5,
    min_lr=1e-4,
)
```

In [28]:

```
history_01 = model_01.fit(
    train_generator,
    steps_per_epoch=train_generator.samples // train_generator.batch_size,
    epochs=50,
    callbacks=[es, cp, lrr],
    validation_data=valid_generator,
    validation_steps=valid_generator.samples // valid_generator.batch_size)
```

```
Epoch 1/50
155/155 ━━━━━━━━━━━━━━━━━━━━ 85s 535ms/step - accuracy: 0.8187 - loss: 19.6773 -
val_accuracy: 0.8750 - val_loss: 8.7893 - learning_rate: 1.0000e-04
Epoch 2/50
155/155 ━━━━━━━━━━━━━━━━━━━━ 2s 8ms/step - accuracy: 0.8438 - loss: 9.0954 - val_
accuracy: 0.8125 - val_loss: 8.7207 - learning_rate: 1.0000e-04
Epoch 3/50
155/155 ━━━━━━━━━━━━━━━━━━━━ 82s 529ms/step - accuracy: 0.9018 - loss: 7.2755 - v
al_accuracy: 0.7500 - val_loss: 4.8408 - learning_rate: 1.0000e-04
Epoch 4/50
155/155 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - accuracy: 0.9062 - loss: 4.5482 - val_
accuracy: 0.7500 - val_loss: 4.9400 - learning_rate: 1.0000e-04
Epoch 5/50
155/155 ━━━━━━━━━━━━━━━━━━━━ 84s 540ms/step - accuracy: 0.9148 - loss: 3.9617 - v
al_accuracy: 0.9375 - val_loss: 2.9452 - learning_rate: 1.0000e-04
Epoch 6/50
155/155 ━━━━━━━━━━━━━━━━━━━━ 1s 6ms/step - accuracy: 0.8125 - loss: 3.0365 - val_
accuracy: 0.8750 - val_loss: 2.9268 - learning_rate: 1.0000e-04
Epoch 7/50
155/155 ━━━━━━━━━━━━━━━━━━━━ 86s 555ms/step - accuracy: 0.9183 - loss: 2.6428 - v
al_accuracy: 0.8125 - val_loss: 2.2993 - learning_rate: 1.0000e-04
Epoch 8/50
155/155 ━━━━━━━━━━━━━━━━━━━━ 15s 97ms/step - accuracy: 0.9688 - loss: 1.9852 - va
l_accuracy: 0.8125 - val_loss: 2.2886 - learning_rate: 1.0000e-04
Epoch 9/50
155/155 ━━━━━━━━━━━━━━━━━━━━ 79s 507ms/step - accuracy: 0.9253 - loss: 1.9280 - v
al_accuracy: 0.7500 - val_loss: 1.7985 - learning_rate: 1.0000e-04
Epoch 10/50
155/155 ━━━━━━━━━━━━━━━━━━━━ 1s 6ms/step - accuracy: 0.9375 - loss: 1.5513 - val_
accuracy: 0.8125 - val_loss: 1.7795 - learning_rate: 1.0000e-04
Epoch 11/50
155/155 ━━━━━━━━━━━━━━━━━━━━ 81s 518ms/step - accuracy: 0.9256 - loss: 1.5076 - v
al_accuracy: 0.8125 - val_loss: 1.4459 - learning_rate: 1.0000e-04
Epoch 12/50
155/155 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - accuracy: 0.9062 - loss: 1.3249 - val_
accuracy: 0.8125 - val_loss: 1.4481 - learning_rate: 1.0000e-04
Epoch 13/50
155/155 ━━━━━━━━━━━━━━━━━━━━ 81s 523ms/step - accuracy: 0.9313 - loss: 1.2018 - v
al_accuracy: 0.7500 - val_loss: 1.4300 - learning_rate: 1.0000e-04
Epoch 14/50
155/155 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - accuracy: 0.9688 - loss: 1.0129 - val_
accuracy: 0.7500 - val_loss: 1.4969 - learning_rate: 1.0000e-04
Epoch 15/50
155/155 ━━━━━━━━━━━━━━━━━━━━ 81s 521ms/step - accuracy: 0.9293 - loss: 1.0033 - v
al_accuracy: 0.8125 - val_loss: 1.0627 - learning_rate: 1.0000e-04
Epoch 16/50
155/155 ━━━━━━━━━━━━━━━━━━━━ 2s 7ms/step - accuracy: 0.9688 - loss: 0.9470 - val_
accuracy: 0.7500 - val_loss: 1.0176 - learning_rate: 1.0000e-04
Epoch 17/50
155/155 ━━━━━━━━━━━━━━━━━━━━ 81s 522ms/step - accuracy: 0.9210 - loss: 0.8734 - v
al_accuracy: 0.7500 - val_loss: 0.8756 - learning_rate: 1.0000e-04
Epoch 18/50
155/155 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - accuracy: 0.8438 - loss: 0.8499 - val_
accuracy: 0.8125 - val_loss: 0.9366 - learning_rate: 1.0000e-04
Epoch 19/50
155/155 ━━━━━━━━━━━━━━━━━━━━ 80s 516ms/step - accuracy: 0.9252 - loss: 0.7430 - v
al_accuracy: 0.7500 - val_loss: 1.1754 - learning_rate: 1.0000e-04
Epoch 20/50
155/155 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 1.0000 - loss: 0.5277 - val_
accuracy: 0.7500 - val_loss: 1.2828 - learning_rate: 1.0000e-04
```

```
Epoch 21/50
155/155 ━━━━━━━━━━━━━━━━━━━━ 82s 524ms/step - accuracy: 0.9273 - loss: 0.6528 - v
al_accuracy: 0.7500 - val_loss: 0.7349 - learning_rate: 1.0000e-04
Epoch 22/50
155/155 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - accuracy: 1.0000 - loss: 0.4878 - val_
accuracy: 0.7500 - val_loss: 0.7428 - learning_rate: 1.0000e-04
Epoch 23/50
155/155 ━━━━━━━━━━━━━━━━━━━━ 81s 519ms/step - accuracy: 0.9341 - loss: 0.5793 - v
al_accuracy: 0.7500 - val_loss: 1.2050 - learning_rate: 1.0000e-04
Epoch 24/50
155/155 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - accuracy: 0.9688 - loss: 0.4676 - val_
accuracy: 0.7500 - val_loss: 1.2424 - learning_rate: 1.0000e-04
Epoch 25/50
155/155 ━━━━━━━━━━━━━━━━━━━━ 115s 739ms/step - accuracy: 0.9332 - loss: 0.5230 -
val_accuracy: 0.8125 - val_loss: 0.6969 - learning_rate: 1.0000e-04
Epoch 26/50
155/155 ━━━━━━━━━━━━━━━━━━━━ 3s 6ms/step - accuracy: 0.9688 - loss: 0.4717 - val_
accuracy: 0.8125 - val_loss: 0.7089 - learning_rate: 1.0000e-04
Epoch 27/50
155/155 ━━━━━━━━━━━━━━━━━━━━ 208s 1s/step - accuracy: 0.9381 - loss: 0.4671 - val
_accuracy: 0.7500 - val_loss: 0.7169 - learning_rate: 1.0000e-04
Epoch 28/50
155/155 ━━━━━━━━━━━━━━━━━━━━ 1s 7ms/step - accuracy: 1.0000 - loss: 0.3236 - val_
accuracy: 0.7500 - val_loss: 0.6124 - learning_rate: 1.0000e-04
Epoch 29/50
155/155 ━━━━━━━━━━━━━━━━━━━━ 238s 2s/step - accuracy: 0.9313 - loss: 0.4434 - val
_accuracy: 0.7500 - val_loss: 0.8966 - learning_rate: 1.0000e-04
Epoch 30/50
155/155 ━━━━━━━━━━━━━━━━━━━━ 3s 7ms/step - accuracy: 1.0000 - loss: 0.3514 - val_
accuracy: 0.7500 - val_loss: 0.9352 - learning_rate: 1.0000e-04
Epoch 31/50
155/155 ━━━━━━━━━━━━━━━━━━━━ 103s 655ms/step - accuracy: 0.9321 - loss: 0.3992 -
val_accuracy: 0.7500 - val_loss: 1.0728 - learning_rate: 1.0000e-04
Epoch 32/50
155/155 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.8750 - loss: 0.4332 - val_
accuracy: 0.7500 - val_loss: 1.1002 - learning_rate: 1.0000e-04
Epoch 32: early stopping
```

In [ ]:
```
model_01_path = MODELS_DIR / "vgg16_model_01.h5"
model_01.save(filepath=model_01_path, overwrite=True)
print(f"Model saved to: {model_01_path}")
```

```
WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `ker
as.saving.save_model(model)`. This file format is considered legacy. We recommend
using instead the native Keras format, e.g. `model.save('my_model.keras')` or `ke
ras.saving.save_model(model, 'my_model.keras')`.
Model saved to: C:\Users\user\Pneumonia_Detection\models\vgg16_model_011.h5
```

In [30]:
```
test_loss, test_accuracy = model_01.evaluate(test_generator)
print(f"Test Accuracy: {test_accuracy}")
```

```
16/16 ━━━━━━━━━━━━━━━━━━━━ 8s 478ms/step - accuracy: 0.7091 - loss: 0.9833
Test Accuracy: 0.8037189841270447
```

In [ ]:
```
model_01.load_weights(MODELS_DIR / "vgg16_model_01.h5")

vgg_val_eval_01 = model_01.evaluate(valid_generator)
vgg_test_eval_01 = model_01.evaluate(test_generator)
```

```
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 382ms/step - accuracy: 0.7500 - loss: 1.1002
16/16 ━━━━━━━━━━━━━━━━━━━━ 8s 483ms/step - accuracy: 0.7091 - loss: 0.9833
```

In [32]:
```
print(f"Validation Loss: {vgg_val_eval_01[0]}")
print(f"Validation Accuarcy: {vgg_val_eval_01[1]}")
print(f"Test Loss: {vgg_test_eval_01[0]}")
print(f"Test Accuarcy: {vgg_test_eval_01[1]}")
```

```
Validation Loss: 1.1002418994903564
Validation Accuarcy: 0.75
Test Loss: 0.7463323473930359
Test Accuarcy: 0.8037189841270447
```

6. Incremental unfreezing and fine-tuning

In [33]:
```
base_model = VGG16(include_top=False, input_shape=(128, 128, 3), weights="imagen

for layer in base_model.layers:
    layer.trainable = False

x = base_model.output
flat = Flatten()(x)
class_1 = Dense(1024, activation="relu")(flat)
dropout = Dropout(0.5)(class_1)
class_2 = Dense(512, activation="relu")(dropout)
output = Dense(2, activation="softmax")(class_2)

model_02 = Model(base_model.inputs, output)

model_02.load_weights(MODELS_DIR / "vgg16_model_011.h5")

for layer in base_model.layers:
    if layer.name in ["block5_conv1", "block5_conv2", "block5_conv3"]:
        layer.trainable = True

model_02.compile(
    optimizer=Adam(learning_rate=1e-5),
    loss="categorical_crossentropy",
    metrics=["accuracy"]
)

base_model_layer_names = [layer.name for layer in base_model.layers]
model_02.summary()
```

Model: "functional_1"

Layer (type)	Output Shape	Param #
input_layer_1 (InputLayer)	(None, 128, 128, 3)	0
block1_conv1 (Conv2D)	(None, 128, 128, 64)	1,792
block1_conv2 (Conv2D)	(None, 128, 128, 64)	36,928
block1_pool (MaxPooling2D)	(None, 64, 64, 64)	0
block2_conv1 (Conv2D)	(None, 64, 64, 128)	73,856
block2_conv2 (Conv2D)	(None, 64, 64, 128)	147,584
block2_pool (MaxPooling2D)	(None, 32, 32, 128)	0
block3_conv1 (Conv2D)	(None, 32, 32, 256)	295,168
block3_conv2 (Conv2D)	(None, 32, 32, 256)	590,080
block3_conv3 (Conv2D)	(None, 32, 32, 256)	590,080
block3_pool (MaxPooling2D)	(None, 16, 16, 256)	0
block4_conv1 (Conv2D)	(None, 16, 16, 512)	1,180,160
block4_conv2 (Conv2D)	(None, 16, 16, 512)	2,359,808
block4_conv3 (Conv2D)	(None, 16, 16, 512)	2,359,808
block4_pool (MaxPooling2D)	(None, 8, 8, 512)	0
block5_conv1 (Conv2D)	(None, 8, 8, 512)	2,359,808
block5_conv2 (Conv2D)	(None, 8, 8, 512)	2,359,808
block5_conv3 (Conv2D)	(None, 8, 8, 512)	2,359,808
block5_pool (MaxPooling2D)	(None, 4, 4, 512)	0
flatten_1 (Flatten)	(None, 8192)	0
dense_3 (Dense)	(None, 1024)	8,389,632
dropout_1 (Dropout)	(None, 1024)	0
dense_4 (Dense)	(None, 512)	524,800
dense_5 (Dense)	(None, 2)	1,026

Total params: 23,630,146 (90.14 MB)

Trainable params: 15,994,882 (61.02 MB)

Non-trainable params: 7,635,264 (29.13 MB)

In [34]:
```
base_model_layer_names[-8:]
```

Out[34]:
```
['block4_conv1',
 'block4_conv2',
 'block4_conv3',
 'block4_pool',
 'block5_conv1',
 'block5_conv2',
 'block5_conv3',
 'block5_pool']
```

In [35]:
```
history = model_02.fit(
    train_generator,
    steps_per_epoch=train_generator.samples // train_generator.batch_size,
    epochs=20,
    validation_data=valid_generator,
    validation_steps=valid_generator.samples // valid_generator.batch_size,
    callbacks=[es, cp, lrr]
)
```

```
Epoch 1/20
155/155 ━━━━━━━━━━━━━━━━━━━━ 110s 698ms/step - accuracy: 0.9335 - loss: 0.1655 -
val_accuracy: 0.7500 - val_loss: 0.3644 - learning_rate: 1.0000e-05
Epoch 2/20
155/155 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - accuracy: 0.9375 - loss: 0.4507 - val_
accuracy: 0.7500 - val_loss: 0.3851 - learning_rate: 1.0000e-05
Epoch 3/20
155/155 ━━━━━━━━━━━━━━━━━━━━ 91s 585ms/step - accuracy: 0.9467 - loss: 0.1452 - v
al_accuracy: 0.7500 - val_loss: 0.5260 - learning_rate: 1.0000e-05
Epoch 4/20
155/155 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - accuracy: 1.0000 - loss: 0.0652 - val_
accuracy: 0.7500 - val_loss: 0.5416 - learning_rate: 1.0000e-05
Epoch 5/20
155/155 ━━━━━━━━━━━━━━━━━━━━ 94s 603ms/step - accuracy: 0.9550 - loss: 0.1054 - v
al_accuracy: 0.8125 - val_loss: 0.3471 - learning_rate: 1.0000e-05
Epoch 6/20
155/155 ━━━━━━━━━━━━━━━━━━━━ 2s 9ms/step - accuracy: 0.9375 - loss: 0.1373 - val_
accuracy: 0.8125 - val_loss: 0.3334 - learning_rate: 1.0000e-05
Epoch 7/20
155/155 ━━━━━━━━━━━━━━━━━━━━ 93s 595ms/step - accuracy: 0.9554 - loss: 0.1211 - v
al_accuracy: 0.7500 - val_loss: 0.5767 - learning_rate: 1.0000e-05
Epoch 8/20
155/155 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - accuracy: 0.9688 - loss: 0.0972 - val_
accuracy: 0.7500 - val_loss: 0.5744 - learning_rate: 1.0000e-05
Epoch 9/20
155/155 ━━━━━━━━━━━━━━━━━━━━ 94s 607ms/step - accuracy: 0.9597 - loss: 0.0977 - v
al_accuracy: 0.7500 - val_loss: 0.3325 - learning_rate: 1.0000e-05
Epoch 10/20
155/155 ━━━━━━━━━━━━━━━━━━━━ 2s 8ms/step - accuracy: 0.9688 - loss: 0.1129 - val_
accuracy: 0.8125 - val_loss: 0.2723 - learning_rate: 1.0000e-05
Epoch 11/20
155/155 ━━━━━━━━━━━━━━━━━━━━ 92s 592ms/step - accuracy: 0.9661 - loss: 0.0903 - v
al_accuracy: 0.7500 - val_loss: 0.5468 - learning_rate: 1.0000e-05
Epoch 12/20
155/155 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.9688 - loss: 0.0758 - val_
accuracy: 0.7500 - val_loss: 0.4771 - learning_rate: 1.0000e-05
Epoch 13/20
155/155 ━━━━━━━━━━━━━━━━━━━━ 95s 615ms/step - accuracy: 0.9657 - loss: 0.0941 - v
al_accuracy: 0.8750 - val_loss: 0.2268 - learning_rate: 1.0000e-05
Epoch 14/20
155/155 ━━━━━━━━━━━━━━━━━━━━ 2s 8ms/step - accuracy: 1.0000 - loss: 0.0462 - val_
accuracy: 0.9375 - val_loss: 0.1701 - learning_rate: 1.0000e-05
Epoch 15/20
155/155 ━━━━━━━━━━━━━━━━━━━━ 92s 589ms/step - accuracy: 0.9616 - loss: 0.0904 - v
al_accuracy: 0.7500 - val_loss: 0.4714 - learning_rate: 1.0000e-05
Epoch 16/20
155/155 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.9688 - loss: 0.1117 - val_
accuracy: 0.7500 - val_loss: 0.4179 - learning_rate: 1.0000e-05
Epoch 17/20
155/155 ━━━━━━━━━━━━━━━━━━━━ 92s 594ms/step - accuracy: 0.9724 - loss: 0.0801 - v
al_accuracy: 0.9375 - val_loss: 0.1731 - learning_rate: 1.0000e-05
Epoch 18/20
155/155 ━━━━━━━━━━━━━━━━━━━━ 2s 9ms/step - accuracy: 0.9688 - loss: 0.0648 - val_
accuracy: 0.9375 - val_loss: 0.1579 - learning_rate: 1.0000e-05
Epoch 19/20
155/155 ━━━━━━━━━━━━━━━━━━━━ 92s 593ms/step - accuracy: 0.9584 - loss: 0.1059 - v
al_accuracy: 0.8750 - val_loss: 0.2121 - learning_rate: 1.0000e-05
Epoch 20/20
155/155 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.9688 - loss: 0.0606 - val_
accuracy: 0.8750 - val_loss: 0.2295 - learning_rate: 1.0000e-05
```

In [36]:
```
model_02_path = MODELS_DIR / "vgg16_model_022.h5"
model_02.save(filepath=model_02_path, overwrite=True)
print(f"Model saved to: {model_02_path}")
```

```
WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `ker
as.saving.save_model(model)`. This file format is considered legacy. We recommend
using instead the native Keras format, e.g. `model.save('my_model.keras')` or `ke
ras.saving.save_model(model, 'my_model.keras')`.
Model saved to: C:\Users\user\Pneumonia_Detection\models\vgg16_model_022.h5
```

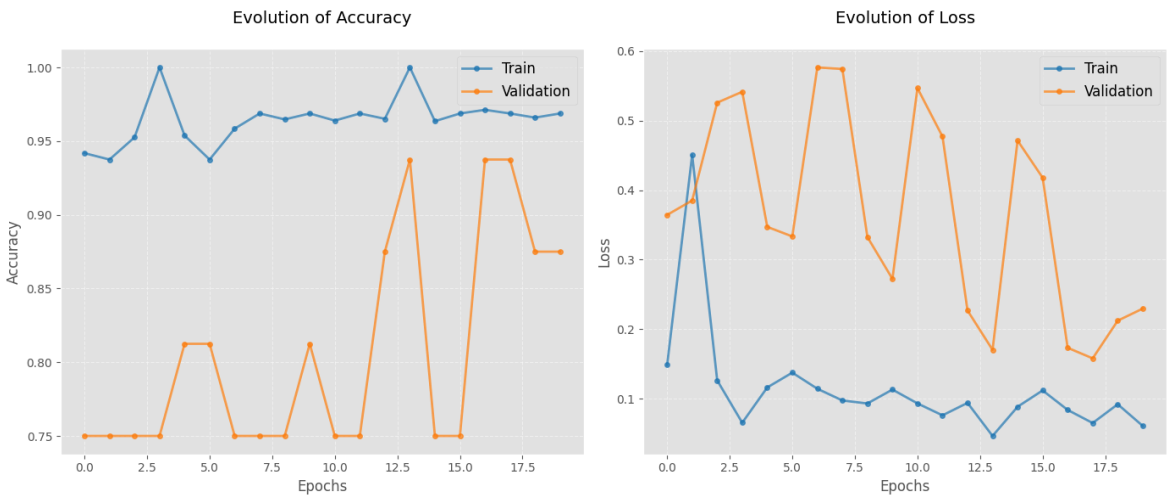
In [37]:
```
model_02.load_weights(MODELS_DIR / "vgg16_model_022.h5")

vgg_val_eval_02 = model_02.evaluate(valid_generator)
vgg_test_eval_02 = model_02.evaluate(test_generator)

print(f"Validation Loss: {vgg_val_eval_02[0]}")
print(f"Validation Accuracy: {vgg_val_eval_02[1]}")
print(f"Test Loss: {vgg_test_eval_02[0]}")
print(f"Test Accuracy: {vgg_test_eval_02[1]}")
```

```
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 370ms/step - accuracy: 0.8750 - loss: 0.2295
16/16 ━━━━━━━━━━━━━━━━━━━━ 8s 443ms/step - accuracy: 0.8470 - loss: 0.4392
Validation Loss: 0.22949431836605072
Validation Accuracy: 0.875
Test Loss: 0.3355826735496521
Test Accuracy: 0.8863636255264282
```

7. Learning curves

In [38]:
```
plt.style.use("ggplot")

plot_kwargs = {"linewidth": 2, "marker": "o", "markersize": 4, "alpha": 0.7}

plt.figure(figsize=(14, 6))

plt.subplot(1, 2, 1)
plt.plot(history.history["accuracy"], label="Train", color="#1f77b4", **plot_kwa
plt.plot(history.history["val_accuracy"], label="Validation", color="#ff7f0e", *
plt.title("Evolution of Accuracy\n", fontsize=14)
plt.xlabel("Epochs", fontsize=12)
plt.ylabel("Accuracy", fontsize=12)
plt.grid(True, linestyle="--", alpha=0.5)
plt.legend(fontsize=12)

plt.subplot(1, 2, 2)
plt.plot(history.history["loss"], label="Train", color="#1f77b4", **plot_kwargs)
plt.plot(history.history["val_loss"], label="Validation", color="#ff7f0e", **plo
plt.title("Evolution of Loss\n", fontsize=14)
plt.xlabel("Epochs", fontsize=12)
plt.ylabel("Loss", fontsize=12)
plt.grid(True, linestyle="--", alpha=0.5)
plt.legend(fontsize=12)

plt.tight_layout()
curves_path = RESULTS_DIR / "training_curves.png"
plt.savefig(curves_path, dpi=300, bbox_inches="tight")
plt.show()
```

```
print(f"Training curves saved to: {curves_path}")
```




```
Training curves saved to: C:\Users\user\Pneumonia_Detection\notebook\results\trai
ning_curves.png
```

In [39]:
```
print("Expected input shape:", model_02.input_shape)
print("Class indices:", train_generator.class_indices)
```

```
Expected input shape: (None, 128, 128, 3)
Class indices: {'NORMAL': 0, 'PNEUMONIA': 1}
```

In [51]:
```
classes = ['NORMAL', 'PNEUMONIA']

correct = 0
total = 0
y_true = []
y_pred = []

for class_idx, class_name in enumerate(classes):
    class_path = TEST_DIR / class_name

    if not class_path.exists():
        print(f"Dossier manquant : {class_path}")
        continue

    image_files = list(class_path.glob("*.*"))
    print(f"{class_name}: {len(image_files)} images")

    for img_path in image_files:
        try:
            img = image.load_img(img_path, target_size=(128, 128))
            img_array = image.img_to_array(img)
            img_array = np.expand_dims(img_array, axis=0)

            prediction = model_02.predict(img_array, verbose=0)
            predicted_class = np.argmax(prediction)

            y_true.append(class_idx)
            y_pred.append(predicted_class)

            if predicted_class == class_idx:
                correct += 1
            total += 1

        except Exception as e:
```

```
            print(f"Erreur avec {img_path.name}: {e}")

if total > 0:
    print(f"\nAccuracy globale : {correct / total:.4f}")
    print("\nMatrice de confusion :")
    print(confusion_matrix(y_true, y_pred))

    print("\nClassification report :")
    print(classification_report(y_true, y_pred, target_names=classes))
else:
    print("Aucune image test trouvée.")
```

```
NORMAL: 234 images
PNEUMONIA: 250 images

Accuracy globale : 0.7583

Matrice de confusion :
[[132 102]
 [ 15 235]]

Classification report :
              precision    recall  f1-score   support

      NORMAL       0.90      0.56      0.69       234
   PNEUMONIA       0.70      0.94      0.80       250

    accuracy                           0.76       484
   macro avg       0.80      0.75      0.75       484
weighted avg       0.79      0.76      0.75       484
```